Reflection: AKS (Ben)

1. Worst case (about 120 words)

Primes are the worst case input for AKS as it has to run through all the checks. The big one is having to pass all the polynomial checks which grow based on the value of R.

This can be seen by the worst case of primes and the worst case of semi primes. At 28 bits. The prime took 4 seconds to compute and the semi prime only took 5.6s. So while the semi prime makes it to the polynomial check, it fails early on and will only have to compute a limited amount of polynomial checks. Compare to the prime which has to compute all the polynomial checks and you see the vast difference between the two.

Fermat and Miller-Rabin run a few cheap random checks and accept a tiny chance of error, while AKS makes the check complete so it is never wrong, but it pays for that proof with about $\log^2 n$ polynomial exponentiations of size $\log^2 n$, which makes it millions of times slower.

2. Hardest bug or design decision (about 140 words)

A design decision was made for AKS to use 64 bits instead of 128 bits to keep low cost operations low. This resulted in a cliff at 2^{32} and greater numbers. This did not need to be fixed, but was an intentional design decision to keep smaller primes faster at the trade off of larger numbers being slower. This was through the whole aks.cpp file. As all coding work was done with Claude, this decision was part of the original code and was the decided by me to keep in the program as a trade off to be used

3. What you'd do differently (about 120 words)

I would implement a function that would use variable types depending on the prime size to be able to compute primes at the fastest possible rate based on the size. To supplement this, I would use types from u2 to u1024 depending on the prime size. (even though u1024 is unfeasible to compute).

This would allow the project to attempt to compute and prime it wishes up to any reasonable number.

This change would ensure that numeric types are kept to the smallest size possible ensuring that operation times are as small as possible.

As well as this, I would include many smaller optimisations that were not included in the code to make sure it runs faster. These were not included in the current implementation so as to compare the algorithms and not optimisation

4. Something you didn't understand before, and now do (about 120 words)

I had no idea how AKS worked before this project. I just knew it was a primality testing algorithm, but nothing about how it worked.

What made the polynomial testing was talking to Claude and going back and forth. Claude explaining how it works and me asking and confirming what that means, and how it works. It really clicked when i understood how the mod r worked and how it negates most of the polynomial expansion

AKS follows a 5 step process:

1. Check for perfect powers, x^y numbers. So 2^2, 4^8 etc.
2. Calculate a r value. This r value is the ring size that is used in further steps.
3. Checks for small prime factors that would invalidate numbers
4. If all numbers checked go above √n then the number is prime
5. Step 5 tests a prime-only equation, (X + a)^n = X^n + a, for lots of small values of a, using small polynomials, and a prime passes every test, so it has to run them all, which is why it takes almost all of AKS's time.